Emporiqa Chat Assistant for PrestaShop

A shopper types "warm jacket under 100, waterproof" into your store. Your search returns everything with "jacket" in the title. The shopper scrolls, gives up, and leaves.

The Emporiqa AI chatbot for PrestaShop 8 and 9 is an online salesperson that closes sales in your PrestaShop store. The module syncs your product catalog and CMS pages to Emporiqa, embeds the chat widget on your storefront, and exposes endpoints for in-chat cart operations and order tracking.

The chatbot acts like an online salesperson. Shoppers describe what they need (or upload a photo of something they like), it finds matching products from your catalog, handles objections like "too expensive" with alternatives instead of a discount, answers questions from your CMS pages, compares items, and walks them to cart and checkout in 65+ languages.

[image]

[image]

> **Integration overview** · **Full Documentation** · **Live Demo** · **Pricing**

Features

- **Closes sales**: Handles objections like "too expensive" by suggesting alternatives from your catalog, instead of a discount.
- **Visual search**: Shoppers upload a photo in the widget; the chatbot describes it and finds matching products in your synced PrestaShop catalog (no extra config required).

- **Brand-safe answers**: Every reply comes from your synced products and CMS pages, never from training data. Low-confidence questions hand off to your team.
- **Product sync**: Real-time webhook sync of catalog products and combinations (variations). Parent/child relationships, attributes, prices (including quantity-based volume discounts / tier pricing), stock levels, and images are all included, along with PrestaShop's native `condition` (new/used/refurbished), `is_virtual` (digital products), and `available_for_order` (display-only / catalog-mode products) flags.
- **Page sync**: CMS pages synced with per-language content so the assistant can answer support questions from your own content.
- **Chat widget**: Automatically embedded on your storefront in the correct language for the current visitor.
- **In-chat cart**: Shoppers can add, update, remove items, and proceed to checkout directly from the chat.
- **Order tracking**: HMAC-signed order lookup with customer email verification to protect customer data. The response includes order status and items plus shipping details, carrier name, tracking number, and a tracking URL (composed from the carrier's URL template) once the order has shipped.
- **Conversion tracking**: Captures chat session IDs at checkout and reports order completion events for revenue attribution.
- **Multi-language**: Automatic language mapping. All translations are consolidated into single webhook payloads per entity.
- **Multi-shop** / **multi-channel**: Auto-discovers shops and maps each to an Emporiqa channel using a slugified shop name (e.g. "My Shop" → `my-shop`). Products and pages assigned to multiple shops include per-channel links, prices, stock, and languages in a single payload. The channel is always passed to the widget and webhooks.
- **One-click connect**: A signed handshake links your store to your Emporiqa account in one click. No Store ID or Connection Secret to copy across tabs. Manual paste stays available on HTTP sites.
- **Bounded background delivery**: Product, page, and order events queue during the request and flush once at request shutdown, with a 1.5-second hard cap on the synchronous send. Admin saves and CSV imports complete locally; the webhook fires after the response is sent, and the merchant request can never wait longer than 1.5 seconds on a slow Emporiqa.

- **Extensibility hooks**: 7 action hooks for developers to customize sync payloads, cancel syncs, or modify widget behavior.

Requirements

- PrestaShop 8.1+ or 9.x
- PHP 7.4+
- An Emporiqa account. Sign up with no card; $25 of signup credit (~100 free conversations) auto-applied

Installation

1. Download the module from the PrestaShop Addons Marketplace.
2. In your PrestaShop back office, go to **Modules > Module Manager > Upload a Module** and upload `emporiqa.zip`.
3. Click **Configure** on the Emporiqa module.
4. Click **Connect to Emporiqa**. A new tab opens on emporiqa.com. Create a free account (no card required, $25 of signup credit) or sign in if you already have one, then pick the store you want to connect (or create a new one). The module is connected when you return.
5. On the **Sync** tab, click **Send my catalog**. Products, pages, and combinations flow through; the widget appears on your storefront when the first product arrives.

On HTTP, or prefer to paste credentials yourself? Expand **Edit credentials manually** on the Configure page. Paste a **Store ID** and **Connection Secret** from your Emporiqa dashboard under **Settings → Store Integration**. Both flows reach the same place.

For order tracking, copy the **Order Tracking URL** shown on the Configure page and paste it into your Emporiqa dashboard under **Store Integration → Order Tracking** (the URL is also auto-derived by one-click connect on most setups).

Configuration

All settings are managed from the module configuration page (**Modules > Emporiqa > Configure**):

Connection Settings

The recommended path is **Connect to Emporiqa** (one-click handshake, no credentials to paste). For HTTP sites or manual setup, expand **Edit credentials manually**:

Setting	Description	Default
Store ID	Your Emporiqa store identifier (filled automatically by one-click connect)	(none)
Connection Secret	HMAC-SHA256 signing secret (filled automatically by one-click connect)	(none)
Order Tracking URL	Read-only endpoint to paste into your Emporiqa dashboard	auto-generated

Advanced

Setting	Description	Default
Sync Products	Enable real-time product sync	On
Sync Pages	Enable real-time CMS page sync	On
Enabled Languages	Languages included in sync payloads	All active shop languages
Webhook URL	Emporiqa webhook endpoint	`https://emporiqa.com/webhooks/sync/`
Batch Size	Products/pages per webhook request during bulk sync	25

Order tracking (with customer email verification) and in-chat cart operations are always enabled. No configuration needed.

Keeping your catalog in sync

The module pushes product, page, and order changes to Emporiqa automatically as they happen via PrestaShop hooks. Per-product changes such as scheduled promos (SpecificPrice), image edits, and combination edits re-emit the affected product on their own; pure stock/out-of-stock changes emit a compact availability-only update instead of rebuilding the whole product.

Some changes affect the whole catalog (category or brand renames, currency rate refreshes, tax-rate or tax-rules-group edits, cart-rule changes, new languages enabled). Running a synchronous per-product re-sync from those hooks would block the admin request, so the module logs an actionable warning in **Advanced Parameters → Logs** instead and leaves the catalog refresh to a manual run.

Re-run a full sync from the **Sync** tab when:

- You see one of the "catalog-wide change" warnings in the PrestaShop log
- You add a new shop in multi-shop mode (existing products won't carry the new shop's data until something else touches them)
- You import products in bulk from a CSV file (PrestaShop sometimes bypasses standard save hooks during bulk imports)
- A custom script, migration, or another module writes catalog data directly to the database
- Emporiqa was unreachable for an extended period (network outage, planned maintenance, expired credentials)

As a safety net, run a full sync once a week to catch any drift that may have built up from background failures.

Module Structure

```
emporiqa/
├── emporiqa.php                    # Main module class (hoo
├── config.xml                      # Module metadata
├── logo.png                        # Module icon
├── classes/
│   ├── EmporiqaCartHandler.php       # In-chat cart oper
│   ├── EmporiqaChannelResolver.php   # Multi-shop → chan
│   ├── EmporiqaLanguageHelper.php    # Language mapping
│   ├── EmporiqaOrderFormatter.php    # Order payload for
│   ├── EmporiqaPageFormatter.php     # CMS page payload
│   ├── EmporiqaProductFormatter.php  # Product/combinati
│   ├── EmporiqaSignatureHelper.php   # HMAC-SHA256 signi
│   ├── EmporiqaSyncService.php       # Bulk sync orchest
│   └── EmporiqaWebhookClient.php     # HTTP client for w
├── controllers/
│   ├── admin/
│   │   ├── AdminEmporiqaController.php        # Admin me
│   │   └── AdminEmporiqaConnectController.php # One-clic
│   └── front/
│       ├── cartapi.php               # Cart API endpoint
│       └── ordertracking.php         # Order tracking en
├── views/
│   ├── css/admin.css                 # Admin configurati
│   ├── img/                          # Module images (re
│   ├── js/
│   │   ├── admin-sync.js            # Bulk sync UI with
│   │   └── front-cart-handler.js    # Chat widget cart i
│   └── templates/
│       ├── admin/configure.tpl       # Configuration pag
│       ├── admin/sync_tab.tpl        # Sync tab template
│       └── hook/header.tpl           # Widget embed (dis
├── translations/                     # Translation catal
└── upgrade/                          # Version upgrade s
```

How It Works

Webhook Sync

When a product or CMS page is created, updated, or deleted in PrestaShop, the module records the change in a per-request map and registers a single `register_shutdown_function`. At shutdown, the module reads the final DB state and emits one webhook per touched entity, with a 1.5-second hard cap on the HTTP call (500ms connect, 1500ms total). The shutdown timing means the merchant's admin or checkout response is sent first (under PHP-FPM via `fastcgi_finish_request` where available); the webhook fires afterwards, capped, so the merchant request can never wait longer than 1.5 seconds even if Emporiqa is unreachable.

All webhooks are signed with HMAC-SHA256 via the `X-Webhook-Signature` header for payload integrity verification.

Product Combinations

PrestaShop products with combinations are synced with their full variation structure. The parent product carries the shared name, description, and images, while each combination carries its specific attributes (size, color, etc.), price, and stock. The assistant understands "this jacket comes in blue and red, sizes S through XL."

The full product (and combination) payload also includes a few PrestaShop-native merchandising and pricing fields so the assistant can describe and sell products accurately:

- `condition`: string or null; PrestaShop's product `condition` (`"new"`, `"used"`, or `"refurbished"`).
- `is_virtual`: boolean; true for digital products with no shipping.
- `available_for_order`: boolean; false for display-only / catalog-mode products. The assistant still describes these but won't add them to the cart.
- `max_order_quantities`: per-channel dict (`{channel: int|null}`) of the maximum allowed per-order quantity. PrestaShop has no native per-order maximum, so this currently always ships `null` (no limit). The field is included for cross-platform contract parity, so a future custom source can populate it.

- `tier_prices`: per-currency list of quantity-based volume discounts (`[{min_quantity, price}]`), present on a price entry only when the product or combination has PrestaShop quantity discounts configured. Each tier reflects the public (guest) shopper's unit price at that break, so the assistant can quote "X each at 10+". Group-, customer-, or country-restricted (B2B) tiers are intentionally excluded.

These flags are part of the full product and combination payload, not the lightweight `product.availability` event. Pure stock/availability changes skip the full rebuild and send a compact `product.availability` event carrying only the identification number, SKU, per-channel availability statuses, and stock quantities, one entry per simple product or per combination.

Multi-Language

Each active shop language is mapped to a standard language code. A single product with translations in 3 languages is sent as one webhook payload with all translations nested: fewer HTTP requests, consistent data.

Registered PrestaShop Hooks

Hook	Purpos
`displayHeader`	Embeds the cha on the storefront
`actionProductSave`	Syncs product o update
`actionProductDelete`	Sends delete ev product and its v
`actionObjectCombination{Add,Update,Delete}After`	Syncs parent pr when combinatio change

`actionObjectCms{Add,Update,Delete}After`	Syncs CMS pag create/update/d
`actionValidateOrder`	Captures chat s and sends order.completed
`actionOrderStatusPostUpdate`	Sends order.cor late payment ca
`actionUpdateQuantity`	Emits a lightweig `product.avai` event when stoc (no full product r
`actionProductOutOfStock`	Emits a `product.avai` event on stock-b transitions
`actionObjectSpecificPrice{Add,Update,Delete}After`	Re-syncs the aff product on sche promos, per-gro reductions, and based volume d (tier pricing)
`actionObjectImage{Add,Update,Delete}After`	Re-syncs the aff product when pr images change
`actionObjectCategory{Update,Delete}After`	Logs an actiona warning so the r can run a full sy (catalog-wide im
`actionObjectManufacturer{Update,Delete}After`	Logs an actiona warning so the r can run a full sy (catalog-wide im

`actionObjectCartRule{Add,Update,Delete}After`	Logs an actiona warning so the r can run a full sy (catalog-wide im
`actionObjectCurrencyUpdateAfter`	Logs an actiona warning so the r can run a full sy (catalog-wide pr
`actionObjectTaxUpdateAfter` / `actionObjectTaxRulesGroupUpdateAfter`	Logs an actiona warning so the r can run a full sy (catalog-wide pr
`actionObjectLanguageAddAfter`	Logs an actiona warning so the r can run a full sy locale needs ba

Extensibility Hooks

Developers can hook into the sync pipeline to customize payloads or cancel syncs:

Hook	Purpose	Key Parameters
`actionEmporiqaFormatProduct`	Modify product/ variation payload before sending	`&$data`, `$product`, `$event_type`
`actionEmporiqaFormatPage`	Modify page payload before sending	`&$data`, `$page`, `$event_type`

`actionEmporiqaFormatOrder`	Modify order tracking payload	`&$data`, `$order`
`actionEmporiqaShouldSyncProduct`	Conditionally cancel a product sync	`$product`, `$event_type`, `&$should_sync`
`actionEmporiqaShouldSyncPage`	Conditionally cancel a page sync	`$page`, `$event_type`, `&$should_sync`
`actionEmporiqaWidgetParams`	Modify chat widget embed parameters	`&$params`
`actionEmporiqaOrderTracking`	Modify order tracking response	`&$data`, `$order`

Pricing

The module is free. Emporiqa is Pay-as-you-go: $0/month base + $0.25/conversation. New accounts get $25 of signup credit (about 100 conversations on us), no card required at signup. After the credit, the monthly cap defaults to $59 and is customer-adjustable from the billing dashboard. Enterprise option for catalogs over 30,000 products. Full pricing at emporiqa.com/pricing/.

Emporiqa also works with Drupal Commerce, WooCommerce, Magento, Shopware, Sylius, and any store via webhook API. One Emporiqa account and dashboard runs across all of them.

Documentation & Support

- **Integration overview**: https://emporiqa.com/integrations/

prestashop/
- **Full documentation**: https://emporiqa.com/docs/prestashop/ (configuration details, webhook format reference, hook examples, troubleshooting)
- **Email**: support@emporiqa.com

License

Academic Free License 3.0 (AFL-3.0)